

PROGRAMAÇÃO DE BANCO DE DADOS

Prof. Rafael Tápio

INTRODUÇÃO A PROGRAMAÇÃO EM BANCOS DE DADOS

Unidade 1 - Seções 1 e 2

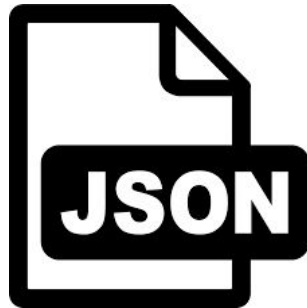
RELEMBRANDO

SISTEMA GERENCIADOR DE BANCO DE DADOS

ARMAZENAMENTO DE DADOS

- Conjunto de dados/informações
- Dados duráveis e acessíveis
- Dados armazenados de maneira organizada

ARMAZENAMENTO DE DADOS



ARMAZENAMENTO DE DADOS

id,nome,idade,profissao,cidade,uf

1,jose silva,30,analista,jundiai,sp

2,maria souza,29,gerente,louveira,sp

3,joao oliveira,32,suporte,jundiaí,sp

ARMAZENAMENTO DE DADOS

<CLIENTE>

<ID>1</ID>

<NOME>JOSE SILVA</NOME>

<IDADE>30</IDADE>

<PROFISSAO>ANALISTA</PROFISSAO>

<CIDADE>JUNDIAI</CIDADE>

<UF>SP</UF>

</CLIENTE>

<CLIENTE>

<ID>1</ID>

<NOME>JOSE SILVA</NOME>

<IDADE>30</IDADE>

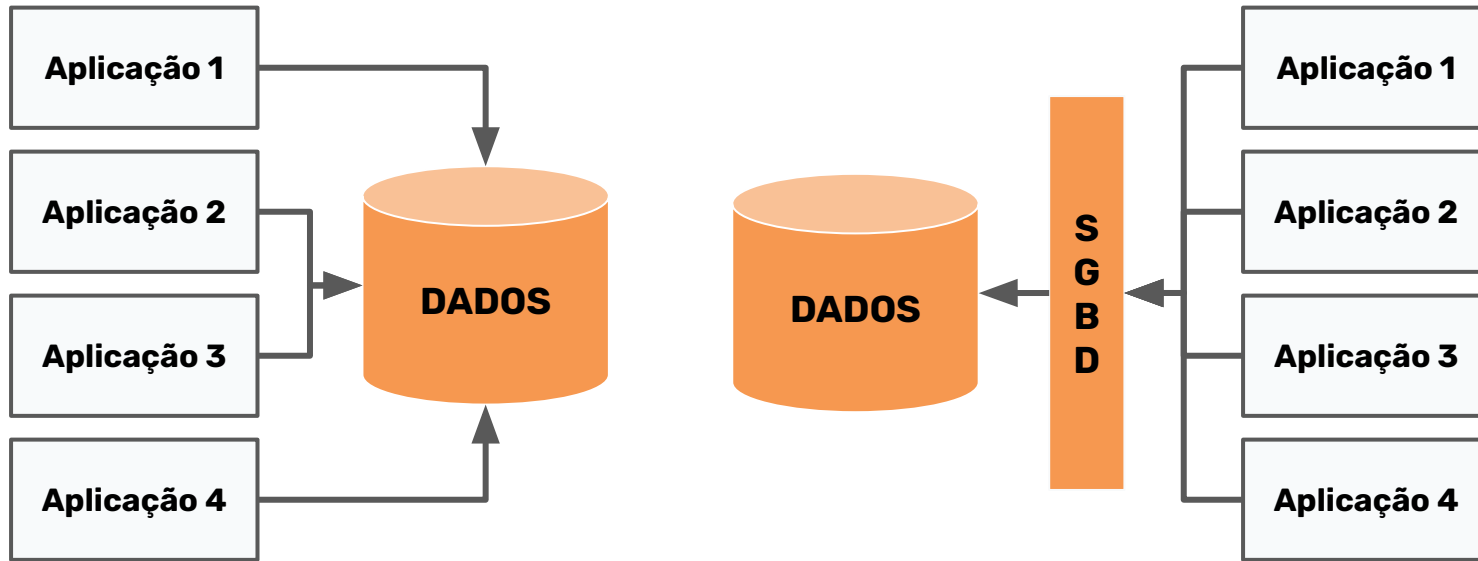
<PROFISSAO>ANALISTA</PROFISSAO>

<CIDADE>JUNDIAI</CIDADE>

<UF>SP</UF>

</CLIENTE>

BANCO DE DADOS X SGBD



SISTEMA GERENCIADOR DE BANCO DE DADOS



SISTEMA GERENCIADOR DE BANCO DE DADOS

- Gerenciar as informações de um banco de dados
- Organizar, acessar, controlar e proteger
- Facilitar a vida do programador

SGBD - COMPONENTES

Tabelas (Tables)

As tabelas são a estrutura fundamental de um banco de dados, onde os dados são armazenados em linhas (registros) e colunas (campos).

Índices (Indexes)

Os índices são estruturas que melhoram a velocidade de recuperação de dados. Eles permitem que o banco de dados encontre rapidamente registros sem precisar verificar cada linha de uma tabela. Índices podem ser criados em uma ou mais colunas de uma tabela.

Views

As views são visões virtuais que apresentam dados de uma ou mais tabelas de forma personalizada. Elas não armazenam dados fisicamente, mas sim uma consulta SQL.

SGBD - COMPONENTES

Stored Procedures

As stored procedures são conjuntos de instruções SQL que são armazenados e podem ser executados no servidor de banco de dados. Elas permitem a execução de operações complexas e de lógica de negócios.

Functions

As functions são semelhantes às stored procedures, mas são usadas para retornar um valor único.

Triggers

Os triggers são conjuntos de instruções SQL que são automaticamente executadas em resposta a certos eventos no banco de dados, como inserções, atualizações ou exclusões de dados.

Sequences

Sequences são objetos de banco de dados usados para gerar números únicos sequenciais. Eles são frequentemente usados para criar valores de chave primária automaticamente.

SGBD - COMPONENTES

Constraints

As constraints são regras aplicadas às tabelas para garantir a integridade dos dados. Existem vários tipos de constraints, incluindo:

- **Primary Key:** Garante que cada registro em uma tabela seja único.
- **Foreign Key:** Mantém a integridade referencial entre tabelas.
- **Unique:** Garante que todos os valores em uma coluna ou grupo de colunas sejam únicos.
- **Check:** Garante que os valores em uma coluna atendam a uma condição específica.

Schemas

Schemas são estruturas organizacionais dentro de um banco de dados que agrupam tabelas, views, procedures e outros objetos de banco de dados.

RELEMBRANDO

TIPOS DE DADOS

Inteiros (INT, INTEGER):

O que são?

Números inteiros (sem casas decimais).

Exemplo de uso:

Ids de usuários, quantidade de produtos.

Ponto flutuante (FLOAT, DOUBLE, DECIMAL):

O que são?

Números com casas decimais

Exemplo de uso:

Preço de produtos, notas de exames.

Texto (VARCHAR, TEXT):

O que são?

Sequências de caracteres (strings).

Exemplo de uso:

Nome de pessoas, endereços, descrições.

Data e hora (DATE, DATETIME, TIMESTAMP):

O que são?

Representam datas e horários.

Exemplo de uso:

Data de nascimento, data de criação de um registro.

Booleano (BOOLEAN):

O que são?

Valores de verdade (verdadeiro ou falso).

Exemplo de uso:

Indicar se um usuário está ativo ou inativo.

Binário (BLOB):

O que são?

Dados binários (imagens, arquivos).

Exemplo de uso:

Armazenar fotos, documentos ou vídeos.

VARIAÇÕES ENTRE BANCOS DE DADOS

MySQL:

TINYINT: Um tipo de dado inteiro muito pequeno (de -128 a 127 ou 0 a 255).

SQL Server:

MONEY: Tipo de dado para armazenar valores monetários.

Oracle:

NUMBER: Tipo de dado numérico altamente flexível, podendo ser configurado para armazenar inteiros, decimais e de ponto flutuante.

CONTROLE DE CONCORRÊNCIA

Introdução

Controle de Concorrência é um mecanismo usado pelos sistemas de gerenciamento de banco de dados (SGBD) para garantir que múltiplos usuários ou processos possam acessar e modificar os dados de forma simultânea, sem causar conflitos ou inconsistências.

Imagine que várias pessoas estão trabalhando em um documento ao mesmo tempo. O controle de concorrência é como se fosse um "**organizador**", que garante que ninguém sobrescreva o trabalho do outro e que todos vejam a versão mais atualizada do documento.

Transações em Banco de Dados

Uma **transação** em um banco de dados é um **conjunto de operações** que são executadas de forma conjunta e que devem ser **completadas com sucesso** ou **não feitas de jeito nenhum**.

É como se fosse um "pacote" de ações, onde todas as etapas precisam ser realizadas sem erros para garantir que o banco de dados continue correto e confiável.

Exemplo: Imagine que você está fazendo uma compra online:

1. Você faz o pedido (primeira operação).
2. O banco de dados registra o pagamento (segunda operação).
3. O estoque é atualizado (terceira operação).

Como funciona?

Leitura e escrita simultânea: Quando várias transações (operações no banco de dados) tentam acessar os mesmos dados ao mesmo tempo, o controle de concorrência evita que elas interfiram umas nas outras.

Isolamento: Ele garante que as transações sejam isoladas, ou seja, o que uma transação está fazendo (por exemplo, atualizando um dado) não vai interferir diretamente nas outras, até que ela seja finalizada.

Sistemas de bloqueio: O banco pode "**bloquear**" os dados enquanto uma transação está sendo realizada, evitando que outras transações façam alterações ao mesmo tempo. Esses bloqueios podem ser de diferentes tipos: **de leitura ou de escrita**.

Exemplos de problemas que ele resolve

Perda de atualização: Quando duas transações tentam atualizar o mesmo dado ao mesmo tempo, uma pode sobrescrever a outra.

Leitura suja: Quando uma transação lê dados que ainda não foram confirmados por outra transação e, mais tarde, esses dados são revertidos.

Inconsistência: Quando as transações causam um estado do banco de dados que não é válido (como uma transação de compra que desconta o estoque mas não debita o valor da conta do cliente).

Exercício: Listando as Transações de uma Operação

Imagine que você está criando um sistema de venda de produtos em um e-commerce. Quando um cliente realiza uma compra, várias ações precisam ser realizadas no banco de dados, e cada uma dessas ações deve ser considerada parte de uma transação para garantir a integridade dos dados.

Tarefa: Liste as transações que ocorrem durante a operação de uma compra no e-commerce.

Exercício: Listando as Transações de uma Operação

- Verificação de estoque:
- Pagamento:
- Atualização de estoque:
- Registro da compra:
- Envio de confirmação:

Como Funciona o Processamento de Transações?

Quando uma transação é iniciada, o banco de dados começa a monitorar todas as operações que são feitas (leitura, gravação, atualização, etc.).

Início da Transação: Quando o cliente faz uma ação (como comprar um produto), o sistema marca o início de uma transação.

Execução das Operações: As ações necessárias (verificação de estoque, processamento de pagamento) são realizadas.

Commit: Se todas as operações ocorrerem sem erro, a transação é finalizada com sucesso (chamado de commit). As mudanças são então permanentes no banco de dados.

Rollback: Se algo der errado em qualquer parte, a transação é desfeita (chamado de rollback), e o banco de dados volta ao estado inicial, como se nada tivesse acontecido.

Exemplo:

1. **Início da Transação:** O sistema começa a registrar todas as ações.
2. **Verificação do Estoque:** O banco de dados checa se o produto está disponível.
3. **Processamento do Pagamento:** O sistema verifica o pagamento do cliente (se aprovado ou não).
4. **Atualização do Estoque:** O produto é retirado do estoque.
5. **Confirmação da Compra:** O sistema envia um e-mail para o cliente confirmando a compra.
6. **Commit ou Rollback:**
 - a. Se tudo ocorrer bem, o sistema faz **commit** e a compra é finalizada.
 - b. Se algo der errado (por exemplo, o pagamento não é aprovado), o sistema faz **rollback** e nenhuma ação é realizada no banco de dados.

LINGUAGEM SQL

O que é SQL?

SQL, ou Structured Query Language (Linguagem de Consulta Estruturada), é uma linguagem de programação usada para comunicar-se com bancos de dados. Com SQL, podemos realizar ações como:

- **Consultar dados:** Buscar informações no banco.
- **Inserir dados:** Adicionar novos registros.
- **Atualizar dados:** Modificar registros existentes.
- **Excluir dados:** Remover registros.

Como funciona Banco de Dados e SQL?

Banco de Dados: Você tem uma **coleção** de **tabelas**. Cada **tabela** armazena dados de forma organizada, como uma **planilha** com **linhas** (**registros**) e **colunas** (**campos**).

Tabelas: Dentro de um banco de dados, os dados são armazenados em **tabelas**. Cada **tabela** tem um **nome** e é composta por **colunas** e **linhas**.

Exemplo Prático

Vamos imaginar que você tem uma tabela chamada alunos, onde armazena informações sobre os estudantes. Ela pode ser assim:

id	nome	idade	nota
1	João	22	7.5
2	Ana	19	8.0
3	Lucas	20	9.0

SELECT (Consultar dados)

O comando SELECT é usado para buscar ou consultar informações do banco de dados.

```
SELECT * FROM alunos;
```

```
SELECT *  
from alunos;
```

INSERT INTO (Inserir dados)

O comando INSERT INTO é usado para adicionar novos registros (linhas) em uma tabela.

```
INSERT INTO alunos (nome, idade, nota)
VALUES ('Maria', 20, 8.5);
```

UPDATE (Atualizar dados)

O comando UPDATE é usado para modificar os dados de um registro já existente.

```
UPDATE alunos  
SET nota = 9  
WHERE nome = 'Maria';
```

DELETE (Excluir dados)

O comando DELETE é usado para remover registros de uma tabela.

```
DELETE FROM alunos  
WHERE nome = 'Maria';
```

Cuidados ao Trabalhar com SQL:

- 1. Evite usar espaços nos nomes de tabelas e campos**
Use underscores (_) para separar palavras (ex: nome_aluno).
- 2. Nomes de tabelas e campos devem ser claros e objetivos**
Seja descritivo! Por exemplo, use alunos ao invés de tab1.
- 3. Não se esqueça do ponto e vírgula (;) no final de cada comando SQL**
O ponto e vírgula marca o final do comando, essencial para que o SGBD saiba onde ele termina.
- 4. Use vírgulas corretamente entre os valores nas instruções**
Erros com vírgulas podem fazer com que o comando falhe.
- 5. Parênteses são importantes!**
Verifique se todos os parênteses estão corretamente abertos e fechados.

Exercício:

Imagine que você tem a tabela de alunos. Tente responder às perguntas usando SQL:

1. Liste todos os alunos.
2. Adicione um aluno chamado "Carlos" com idade 23 e nota 7.0.
3. Atualize a nota de "Ana" para 9.0.
4. Exclua o aluno "Lucas".

CONSULTAS

O que são as Cláusulas **SELECT**, **FROM** e **WHERE**?

- **SELECT**: Usado para selecionar as colunas (campos) que você deseja ver.
- **FROM**: Especifica de qual tabela você quer buscar as informações.
- **WHERE**: Filtra os dados, mostrando apenas os registros que atendem a uma condição específica.

O que são as Cláusulas **SELECT**, **FROM** e **WHERE**?

Imagine que você tem uma tabela chamada alunos, com as seguintes colunas: id, nome, idade, nota.

Você quer fazer perguntas como:

Quais alunos têm mais de 18 anos?

Qual a nota do aluno João?

Essas perguntas podem ser feitas com SQL usando **SELECT**, **FROM** e **WHERE**.

A Cláusula SELECT

A cláusula **SELECT** é usada para selecionar as colunas da tabela que você quer ver.

Se você quiser ver todos os dados da tabela alunos, você usaria:

```
SELECT * FROM alunos;
```

O asterisco () significa todas as colunas.*

Se você quiser ver apenas os nomes dos alunos, você faria:

```
SELECT nome FROM alunos;
```

A Cláusula FROM

A cláusula **FROM** indica de qual tabela você está buscando os dados. Em nossa consulta, estamos usando a tabela alunos.

```
SELECT nome FROM alunos;
```

Isso vai buscar apenas a coluna nome da tabela alunos.

A Cláusula WHERE

A cláusula **WHERE** serve para filtrar os dados e mostrar apenas os registros que atendem a uma condição.

Exemplo 1: Se você quiser ver os alunos com idade maior que 18, você pode fazer:

```
SELECT nome, idade FROM alunos WHERE idade > 18;
```

Exemplo 2: se você quiser encontrar um aluno específico, como o João, você pode fazer:

```
SELECT * FROM alunos WHERE nome = 'João';
```

Combinando SELECT, FROM e WHERE

Agora, vamos combinar as três cláusulas para fazer uma consulta mais completa.

Exemplo 1: Buscar o nome e a nota dos alunos com nota maior que 7:

```
SELECT nome, nota FROM alunos WHERE nota > 7;
```

Exemplo 2: Buscar todos os alunos que têm a nota 10:

```
SELECT * FROM alunos WHERE nota = 10;
```

Exercício:

Você tem a tabela de alunos. Faça as seguintes consultas usando SQL:

1. Liste todos os alunos.
2. Liste os alunos com idade maior que 18.
3. Liste o nome e a nota dos alunos com nota maior que 8.
4. Encontre o aluno com o nome "Ana".
5. Liste todos os alunos que têm nota menor que 7.

EXTENSÕES DE CONSULTAS

AS

O que é Renomeação com o AS em SQL?

Renomeação (AS) permite dar um nome temporário a colunas ou tabelas em uma consulta SQL, para tornar os resultados mais legíveis ou para usar um nome mais conveniente nas operações.

AS é usado para criar um alias, ou seja, um nome alternativo para uma tabela ou coluna.

Usando AS em COLUNAS

Exemplo: Renomeando Coluna

Quando você deseja mostrar uma coluna com um nome mais amigável ou simples:

```
SELECT nome AS aluno_nome FROM alunos;
```

Aqui, a coluna nome será exibida como **aluno_nome** no resultado da consulta.

Usando AS em TABELAS

Renomeando Tabela

Você também pode renomear uma tabela temporariamente em uma consulta:

```
SELECT a.nome, a.idade FROM alunos AS a;
```

Benefícios do Uso de AS:

- **Legibilidade:** Colunas e tabelas com nomes mais claros no resultado.
- **Simplificação:** Facilita o uso de tabelas e colunas longas em consultas complexas.

EXTENSÕES DE CONSULTAS

LIKE

Operações de Strings com LIKE em SQL

O que é o LIKE?

O operador LIKE em SQL é utilizado para buscar padrões em valores de strings (textos) em uma coluna, permitindo filtrar resultados com base em um padrão de texto.

Sintaxe Básica:

```
SELECT * FROM tabela WHERE coluna LIKE 'padrão';
```

Usando Coringas com LIKE:

% : Representa qualquer sequência de caracteres (zero ou mais).

_ : Representa um único caractere.

Operações de Strings com LIKE em SQL

Procurar alunos cujo nome começa com "J":

```
SELECT * FROM alunos WHERE nome LIKE 'J%';
```

Vai buscar nomes como João, Julia, José, etc.

Procurar alunos cujo nome contém "ana" em qualquer parte do nome:

```
SELECT * FROM alunos WHERE nome LIKE '%ana%';
```

Vai buscar nomes como Ana, Joana, Luciana, etc.

Procurar alunos cujo nome tem exatamente 4 letras e começa com "A":

```
SELECT * FROM alunos WHERE nome LIKE 'A____';
```

O ____ (três underscores) significa que o nome deve ter exatamente 4 caracteres, começando com "A".

**EXTENSÕES DE
CONSULTAS**

ORDER BY

Ordenação de Resultados com ORDER BY

O que é o ORDER BY?

A cláusula ORDER BY em SQL é utilizada para ordenar os resultados de uma consulta, seja em ordem crescente ou decrescente.

Sintaxe Básica:

```
SELECT * FROM tabela ORDER BY coluna [ASC | DESC];
```

- **ASC (padrão):** Ordena em ordem crescente.
- **DESC:** Ordena em ordem decrescente.

Ordenação de Resultados com ORDER BY

Ordenar alunos por nome em ordem alfabética crescente:

```
SELECT * FROM alunos ORDER BY nome ASC;
```

Ordenar alunos por idade em ordem decrescente:

```
SELECT * FROM alunos ORDER BY idade DESC;
```

Ordenar por múltiplas colunas:

```
SELECT * FROM alunos ORDER BY nota DESC, nome ASC;
```

EXTENSÕES DE CONSULTAS

GROUP BY

Agrupamento de Resultados com GROUP BY

O que é o GROUP BY?

A cláusula GROUP BY em SQL é usada para agrupar linhas com valores iguais em colunas específicas. Isso é útil quando você deseja resumir ou agrupar dados para aplicar funções agregadas, como SOMA, MÉDIA, MÍNIMO, MÁXIMO, entre outras.

Sintaxe Básica:

```
SELECT coluna, função_agrupada(coluna)
```

```
FROM tabela
```

```
GROUP BY coluna;
```

Agrupamento de Resultados com GROUP BY

Função agregada:

- SUM()
- AVG()
- COUNT()
- MAX()
- MIN()

Agrupamento de Resultados com GROUP BY

Contar quantos alunos existem em cada idade:

```
SELECT idade, COUNT(*) FROM alunos GROUP BY idade;
```

Calcular a média de notas dos alunos por idade:

```
SELECT idade, AVG(nota) FROM alunos GROUP BY idade;
```

Soma das notas por turma (considerando que há uma coluna turma):

```
SELECT turma, SUM(nota) FROM alunos GROUP BY turma;
```

Linguagem de Consulta de Dados (DQL)

O que é DQL?

DQL (Data Query Language) é o conjunto de comandos SQL utilizado para consultar dados em um banco de dados. A principal função do DQL é recuperar informações de uma ou mais tabelas, sem modificar os dados.

O comando mais comum em **DQL** é o **SELECT**.

Estrutura Básica de uma Consulta SQL (DQL):

A estrutura de uma consulta SQL é composta por algumas cláusulas fundamentais. Abaixo, está a estrutura básica:

```
SELECT coluna1, coluna2, ...
```

```
FROM tabela
```

```
WHERE condição
```

```
ORDER BY coluna
```

```
GROUP BY coluna
```

```
HAVING condição_agrupamento;
```

Estrutura Básica de uma Consulta SQL (DQL):

A estrutura de uma consulta SQL é composta por algumas cláusulas fundamentais. Abaixo, está a estrutura básica:

SELECT: Define as colunas que serão retornadas na consulta.

FROM: Especifica a tabela de onde os dados serão recuperados.

WHERE: Filtra os dados de acordo com uma condição.

ORDER BY: Ordena os dados com base em uma ou mais colunas.

GROUP BY: Agrupa os dados por uma ou mais colunas.

HAVING: Filtra os grupos de dados após o agrupamento.

Exemplo de Consulta Completa:

Vamos supor uma tabela de **vendas** com as colunas **id**, **produto**, **quantidade**, **preco**, **data_venda** e **vendedor**.

```
SELECT vendedor, SUM(quantidade) AS total_vendas  
FROM vendas  
WHERE data_venda >= '2025-01-01'  
GROUP BY vendedor  
HAVING SUM(quantidade) > 50  
ORDER BY total_vendas DESC;
```

Exemplo de Consulta Completa:

- **SELECT:** Retorna o vendedor e a soma da quantidade de produtos vendidos (renomeado como total_vendas).
- **FROM:** A consulta é feita na tabela vendas.
- **WHERE:** Filtra os registros onde a data_venda é a partir de 1º de janeiro de 2025.
- **GROUP BY:** Agrupa as vendas por vendedor.
- **HAVING:** Filtra os vendedores que venderam mais de 50 produtos.
- **ORDER BY:** Ordena os vendedores pelo total de vendas em ordem decrescente.

Exercício:

Abaixo está uma consulta SQL. Explique o que cada parte da consulta faz, com base na estrutura de DQL que aprendemos.

```
SELECT cliente, COUNT(*) AS total_compras, SUM(valor_compra) AS total_gasto  
FROM compras  
WHERE data_compra >= '2025-01-01'  
GROUP BY cliente  
HAVING SUM(valor_compra) > 5  
ORDER BY total_gasto DESC;
```